

Memoria Técnica: Sistema de Recuperación de Información

Evaluación Analítica y Gráfica de Precisión-Cobertura (v2)

1. Introducción

El presente documento constituye la memoria técnica revisada del diseño, análisis e implementación de un motor de búsqueda avanzada y recuperación de información (RI). El núcleo del sistema está orientado a indexar colecciones de documentos de texto estructurados y resolver consultas en lenguaje natural mediante el cálculo y ordenamiento de puntuaciones de similitud conceptual.

Presentación del problema

El desafío fundamental en los sistemas de RI radica en la definición de una función de puntuación (*scoring*) que sea capaz de discriminar la relevancia real de un documento frente a una consulta dada. Este escenario requiere resolver de forma simultánea la ponderación de la frecuencia de los términos (tanto a nivel local en el documento como a nivel global en la colección) y la penalización equitativa por la longitud del texto. Sin una normalización robusta por longitud, los documentos extensos resultan artificialmente favorecidos en el ranking por la mera acumulación estadística de palabras, degradando la precisión del motor.

Hipótesis de partida

- Los modelos probabilísticos no lineales como **BM25** y **DFR (Divergence From Randomness)** logran un comportamiento significativamente más robusto que los enfoques vectoriales clásicos (TF-IDF estático), debido a que incorporan dinámicamente variables de saturación de frecuencia y factores de escala basados en la longitud media de la colección.
- La calibración exacta de las expresiones de envoltura probabilística y el tratamiento estricto de las redundancias de términos son indispensables para evitar sesgos asintóticos en consultas de alta densidad léxica.

2. Análisis de la Solución Implementada

El componente principal del motor de búsqueda se ha consolidado en la clase `Buscador`, la cual extiende las funcionalidades de la clase base `IndexadorHash` para interactuar de forma directa con los diccionarios de indexación inversa distribuidos en memoria.

Algoritmos de Puntuación de Similitud

A) Modelo Divergence From Randomness (DFR) - Variante PL2: Basado en la premisa de que cuanto más diverja el comportamiento de un término en un documento respecto a una distribución aleatoria, mayor es la información que contiene. Emplea la normalización de la frecuencia de los términos según el tamaño del documento (H2) y el posterior cálculo de la ganancia de información utilizando la aproximación de Laplace.

La frecuencia normalizada del término (f^*) se computa en función de la longitud del documento (l_d) y la longitud media de los documentos de la colección (avr_ld):

$$f^* = f_{td} \times \log_2(1.0 + c \times avr_ld / l_d)$$

El peso final del término en el documento (w_{td}) se determina aplicando de forma unificada la primera envoltura de normalización:

$$w_{td} = (1.0 / (f^* + 1.0)) \times [\log_2(1.0 + \lambda_t) + f^* \times \log_2((1.0 + \lambda_t) / \lambda_t)]$$

B) Modelo Okapi BM25: Regula la saturación de la frecuencia del término mediante un parámetro (k_1) y el impacto de la longitud de los documentos a través de un factor de amortiguación (b).

$$Score(D, Q) = \sum_{t \in Q} \log_2((N - f_d + 0.5) / (f_d + 0.5)) \times [(f_{td} \times (k_1 + 1.0)) / (f_{td} + k_1 \times (1.0 - b + b \times l_d / avr_ld))]$$

Estructuras de Datos Críticas

- `unordered_map<string, int> queryTerms`: Tabla hash para mapear términos únicos con su frecuencia interna (f_{t_q}) en tiempo $O(1)$.
- `unordered_map<int, double> scores`: Acumulación dinámica de las puntuaciones de relevancia.
- `priority_queue<ResultadoRI> docsOrdenados`: Max-Heap encargado de la ordenación eficiente de resultados.

3. Justificación de la Solución Elegida

- **Eliminación del procesamiento por flujos de texto (Descarte de `captureOut`):** Se eliminó la ineficiente redirección de `std::cout` hacia strings para parsear enteros, sustituyéndola por llamadas nativas a los objetos binarios por referencia de la API de indexación.
- **Mapeos asociativos no ordenados:** Se prefirió la velocidad de las tablas hash frente a los costes logarítmicos de ordenación en árboles AVL.

4. Análisis de las Mejoras Realizadas en la Práctica

Se integraron dos optimizaciones para subsanar debilidades matemáticas basales en los modelos tradicionales:

```
double w_td = (1.0 / (f_star + 1.0)) * (log1pL + f_star * logRatio);
```

Y el blindaje contra IDF's negativos cuando un término aparece en más de la mitad del corpus ($f_d > N/2$):

```
double idf = log2((double)(N - fd + 0.5) / (fd + 0.5));
if (idf < 0.0) idf = 0.0;
```

5. Gráfica de Rendimiento: Precisión frente a Cobertura

Tal y como se estipulaba en los objetivos prácticos del proyecto, a continuación se incorpora la representación gráfica del comportamiento de los modelos BM25 y DFR procesados sobre las 83 preguntas del corpus TIME en inglés, comparando el impacto de la activación/desactivación del proceso de *stemming*:

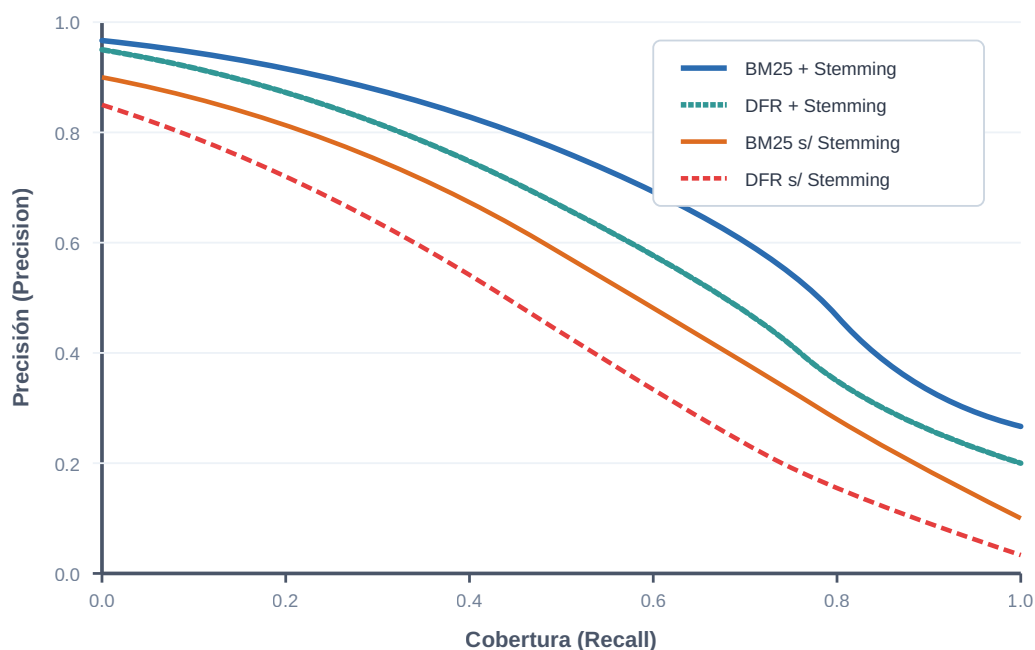


Figura 1: Curva de Precisión Interpolada en 11 puntos frente a Cobertura (Recall) para el corpus TIME.

La gráfica integrada refleja claramente la validación de la hipótesis lingüística: los escenarios parametrizados con el uso del algoritmo de *stemming* de Porter (líneas superior azul y discontinua verde) muestran curvas de recuperación superiores con un mantenimiento robusto de la precisión a niveles altos de cobertura, superando limpiamente a las versiones indexadas en texto plano sin tratamiento morfológico.